Discrete Mathematics

CSE 4203

Dr. Md Azam Hossain

11.3 Tree Traversal

September 4, 2025

Outline

Ordered Rooted Trees and Traversals

- **Ordered rooted trees** are widely used to store and organize information.
- Traversal algorithms allow visiting each vertex systematically to **access or process data**.
- These trees are useful for representing **expressions**, such as:
  - Arithmetic expressions with numbers, variables, and operations.
- Different traversal orders of such trees correspond to different ways of **evaluating expressions**.

Traversal Algorithms (Section 11.3.3)

- **Traversal algorithms** are procedures for systematically visiting every vertex of an **ordered rooted tree**.
- We focus on three commonly used algorithms:
  1. **Preorder Traversal**
  2. **Inorder Traversal**
  3. **Postorder Traversal**
- Each traversal can be defined **recursively**.
- We begin with the definition of **Preorder Traversal**.

Preorder Traversal

Definition:

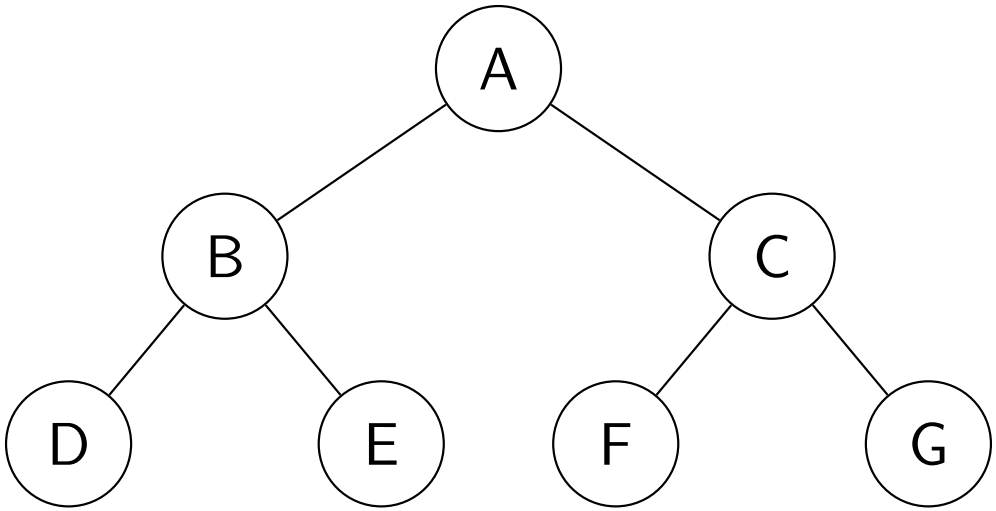
- In **Preorder Traversal**, the root is visited first, followed by recursively visiting all subtrees from left to right.
- Order of visit: **Root** $\rightarrow$ **Left Subtree** $\rightarrow$ **Right Subtree**.

Pseudocode:

```
Preorder(node):
  if node ≠ null:
     visit(node)
     Preorder(node.left)
     Preorder(node.right)
```

Example: Preorder Traversal

Tree Diagram:



Preorder Traversal Order: A, B, D, E, C, F, G

Preorder Traversal Example

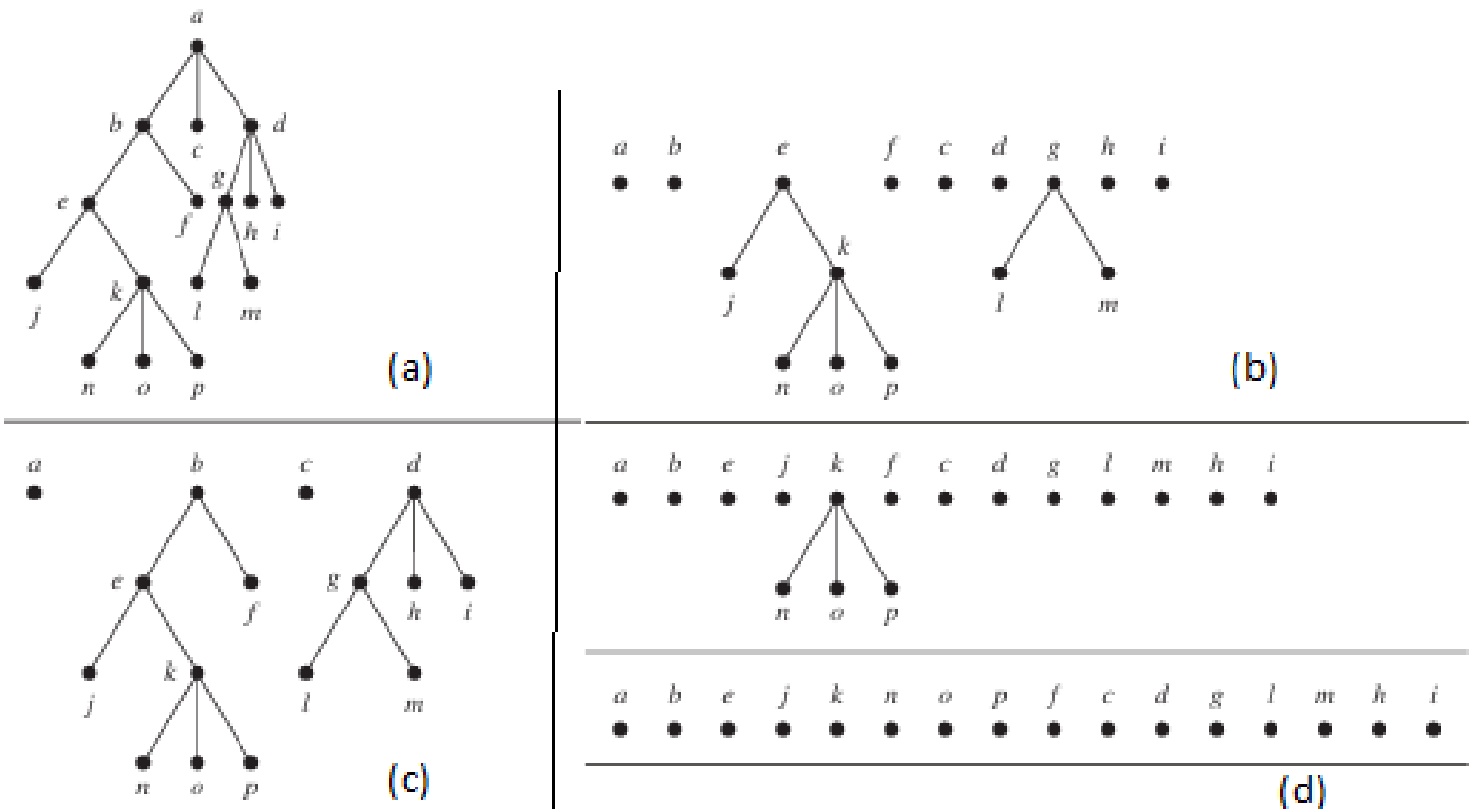


Figure: An example of a preorder traversal.

Inorder Traversal

Definition:

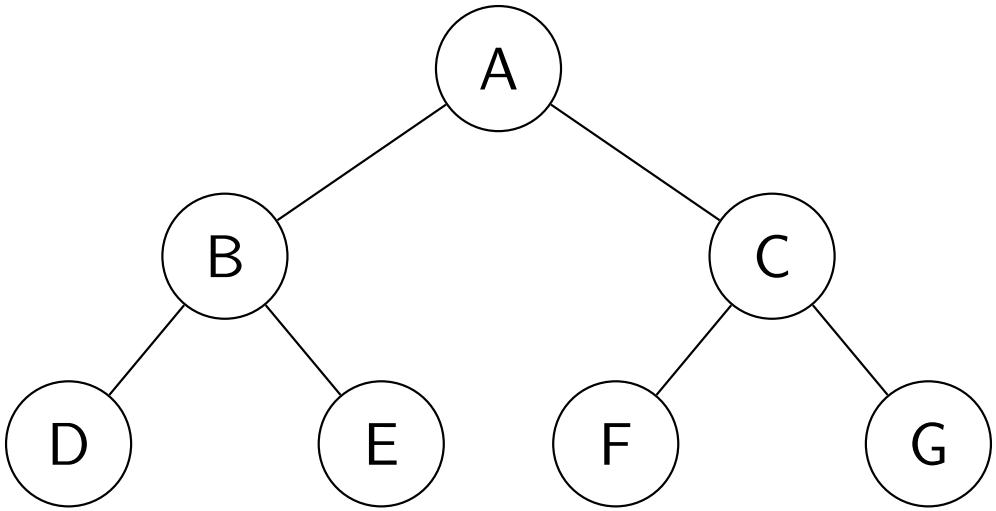
- In **Inorder Traversal**, the left subtree is visited first, then the root, followed by the right subtree.
- Order of visit: **Left Subtree → Root → Right Subtree**.
- Commonly used in **Binary Search Trees** to get sorted order.

Pseudocode:

```
Inorder(node):
  if node ≠ null:
     Inorder(node.left)
     visit(node)
     Inorder(node.right)
```

Example: Inorder Traversal

Tree Diagram:



Inorder Traversal Order: D, B, E, A, F, C, G

Postorder Traversal

Definition:

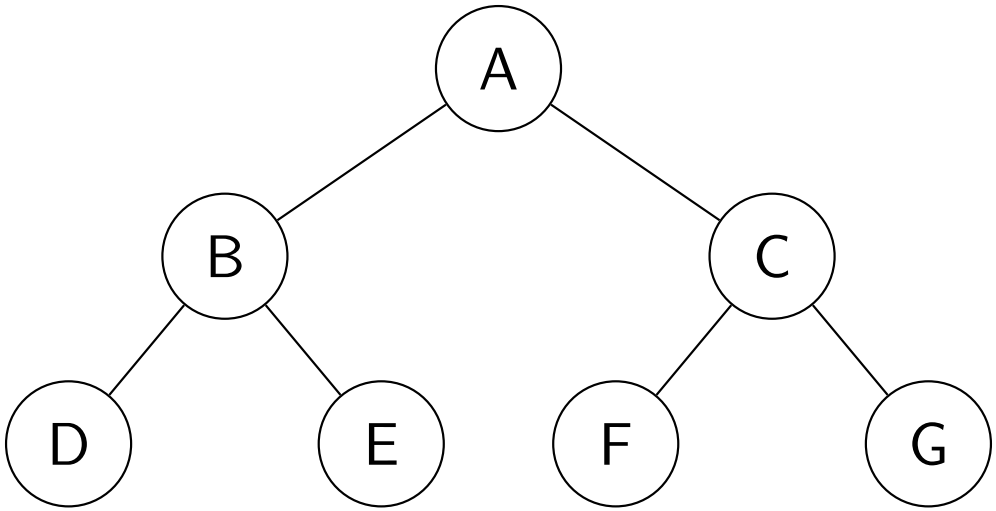
- In **Postorder Traversal**, the left subtree is visited first, then the right subtree, followed by the root.
- Order of visit: **Left Subtree → Right Subtree → Root**.
- Commonly used for **deleting nodes** or **evaluating expression trees**.

Pseudocode:

```
Postorder(node):
  if node ≠ null:
     Postorder(node.left)
     Postorder(node.right)
     visit(node)
```

Example: Postorder Traversal

Tree Diagram:



Postorder Traversal Order: D, E, B, F, G, C, A

Infix, Prefix, and Postfix Notation

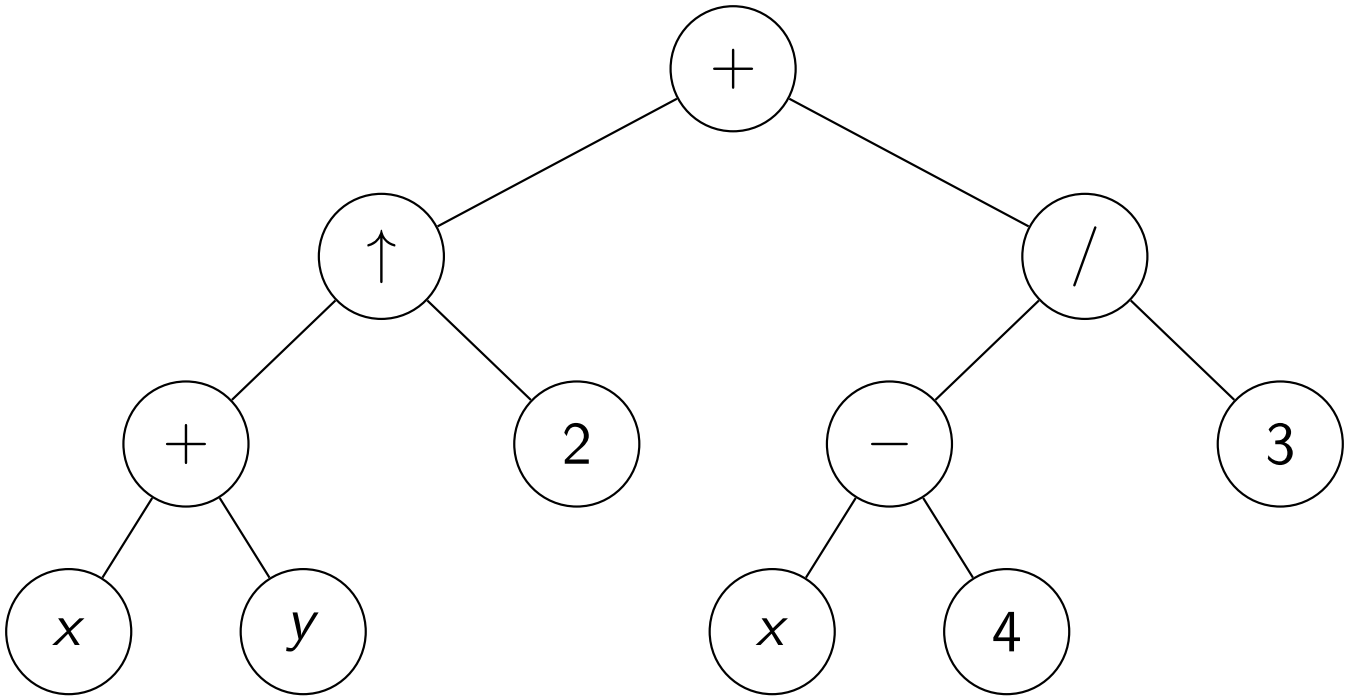
- Ordered rooted trees can represent **complex expressions**, such as:
  - Arithmetic expressions
  - Compound propositions
  - Combinations of sets
- In arithmetic expressions, the internal nodes represent **operators** (+, −, ∗, /, ˆ), and the leaves represent **variables or numbers**.
- Each operation acts on its **left and right subtrees** (in that order).
- Parentheses can indicate the **order of operations**, but the tree structure makes traversal-based evaluation natural.

Example: Expression Tree

Expression:

$$((x + y) \uparrow 2) + ((x - 4)/3)$$

Ordered Rooted Tree Representation:



Note: Internal nodes represent **operators**, leaves represent **variables/numbers**.

Prefix (Polish) Notation

Definition:

- The **prefix form** of an expression is obtained by traversing its expression tree in **preorder**.
- Expressions in prefix notation are also called **Polish notation**, named after Jan Lukasiewicz.
- In prefix form, each operator precedes its operands.
- This notation is **unambiguous**, so **no parentheses** are needed.

Example 6: Convert the expression to prefix form

$$((x + y) \uparrow 2) + ((x - 4)/3)$$

Solution: Traverse the expression tree in **preorder**:

$$\text{Prefix form: } + \ \uparrow \ + \ x \ y \ 2 \ / \ - \ x \ 4 \ 3$$

Postfix (Reverse Polish) Notation

Definition:

- The **postfix form** of an expression is obtained by traversing its expression tree in **postorder**.
- Expressions in postfix notation are also called **Reverse Polish notation**.
- In postfix form, each operator follows its operands.
- This notation is **unambiguous**, so **no parentheses** are needed.

Example: Convert the expression to postfix form

$$((x + y) \uparrow 2) + ((x - 4)/3)$$

Solution: Traverse the expression tree in **postorder**:

$$\text{Postfix form: } x\ y\ +\ 2\ \uparrow\ x\ 4\ -\ 3\ /\ +$$

Infix Notation

Definition:

- The **infix form** of an expression is obtained by traversing its expression tree in **inorder**.
- In infix notation, each operator is written **between its operands**.
- Parentheses are used to indicate the **order of operations** and make the expression unambiguous.

Example: Convert the expression tree to infix form

$$((x + y) \uparrow 2) + ((x - 4)/3)$$

Solution: Traverse the expression tree in **inorder**:

$$\text{Infix form: } ((x + y) \uparrow 2) + ((x - 4)/3)$$

Comparison of Expression Notations

Expression:

$$((x + y) \uparrow 2) + ((x - 4)/3)$$

Notation	Expression Form
Infix	((x + y) ↑ 2) + ((x - 4) / 3)
Prefix	+ ↑ + x y 2 / - x 4 3
Postfix	x y + 2 ↑ x 4 - 3 / +

Notes:

- **Infix:** Operators between operands, parentheses needed.
- **Prefix:** Operators before operands, no parentheses needed.
- **Postfix:** Operators after operands, no parentheses needed.

Evaluating Expressions: Prefix and Postfix

Expression:

$$((x + y) \uparrow 2) + ((x - 4)/3), \quad x = 2,\ y = 3$$

Prefix Form: $+ \uparrow + \text{x y } 2 \ / \ \text{- x } 4\ 3$

Evaluation (Prefix):

1. Compute $+ \text{ x y} \rightarrow 2 + 3 = 5$
2. Compute $\uparrow 5\ 2 \rightarrow 5^2 = 25$
3. Compute $\text{- x } 4 \rightarrow 2 - 4 = \text{-2}$
4. Compute $/ \text{ -2 } 3 \rightarrow \text{-2 } / \ 3 \ \ \text{-0.6667}$
5. Compute $+ \ 25 \ \text{-0.6667} \rightarrow 24.3333$

Postfix Form: $\text{x y} + 2 \uparrow \text{x } 4 \text{ - } 3 \ / \ +$

Evaluation (Postfix):

1. Compute $\text{x y} + \rightarrow 2 + 3 = 5$
2. Compute $5\ 2 \uparrow \rightarrow 5^2 = 25$
3. Compute $\text{x } 4 \text{ - } \rightarrow 2 - 4 = \text{-2}$
4. Compute $\text{-2 } 3 \ / \rightarrow \text{-2 } / \ 3 \ \ \text{-0.6667}$
5. Compute $25 \ \text{-0.6667} + \rightarrow 24.3333$